
# COMP90041
# Programming and Software Development

Lecture 6 – Arrays II

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

THE UNIVERSITY OF MELBOURNE

# Arrays II - Overview


**In this part of the lecture, you will learn about:**

- **Methods with a variable No. of parameters**
- **Resizable arrays**
- **Sorting arrays**
- **Multidimensional arrays**

# Methods with a variable No. of parameters


**What are Varargs?**

- Introduced in Java 5.0
- Allow methods to accept any number of arguments
- Useful when the number of inputs **isn't fixed**
- Syntax

```
Type ... name → e.g.: int... nums
```

  - Vararg must be the **last** parameter
  - Ellipsis **...** is required
- How it works?
  - Java creates automatically an array from the arguments
- Regular parameters still work as usual

# Methods with a variable No. of parameters

- **Example:**

```
public static int max(int... numbers){
    int largest = Integer.MIN_VALUE;
    for (int a : numbers)
        if (a > largest)
            largest = a;
    return largest;
}
```

- int... numbers = **zero or more** int values
- Invocation:

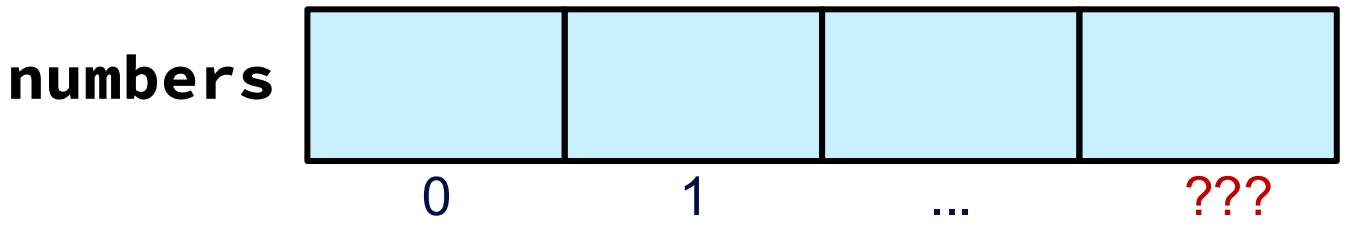
```
max(3, 7, 2);
max();
```

- **Another example** that uses variable numbers of parameters is `System.out.printf`
  - System.out.printf(format, argument1, argument2, ..., argumentN);
  - Example: System.out.printf("Age: %d, Name: %s", 25, "Alice");

# Unknown Array Size

- Sometimes the final size of an array **isn't known** at the start
- Even with dynamic allocation, estimating size can be difficult
- **Example:** Imagine you're asking the user to **enter as many numbers** as they like. You **don't know** in advance how many they'll enter. How do you store these numbers?

```
int[] numbers = new double[???];
```




- If you guess **too small**: you'll **run out of space**
- If you guess **too big**: you **waste** memory

# Unknown Array Size

## Naive solution:

- **Reallocate** the whole array each time a new element is added
- Very **inefficient**: copying overhead increases

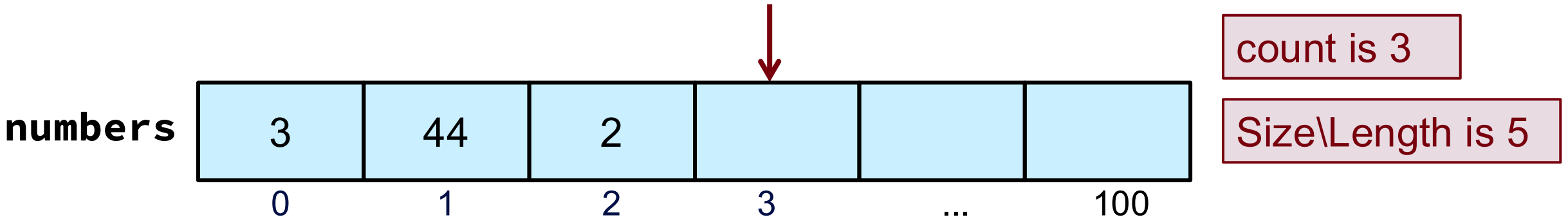
```
int [] numbers = {};
for (int i = 0; i < 100; i++) {
        // Create a new array, 1 element larger
        int [] newArray = new int[numbers.length + 1];

        // Copy existing elements
        for (int j = 0; j < numbers.length; j++) {
                newArray [j] = numbers [j];
        }
        // Add new element
        newArray [numbers.length] = i;
        numbers = newArray;
}
```

Step 1: [] → [3]
Step 2: [3] → [3, 6]
Step 3: [3, 6] → [3, 6, 2]

**4950 times**

# Partially Filled Array

- **Issue:** accessing **unused** elements :
  - Uninitialized values cause **unpredictable** behaviour
  - **Hard to debug** compared to null errors
- **array.length** shows the **total** capacity, not how many elements are filled
- **Solution: Track** how many elements are actually **used** in large array.



# Partially Filled Array

## Size tracking in Fixed array solution:

- **Solution: Track** how many elements are actually **used** in large array.
  - **Count** must be an **argument** to any method that uses the partially filled array
  - **Solution: Encapsulating** Array and Count in a class
  - **Valid** range: numbers[0] to numbers[count - 1]

```
int [] numbers = new int[1000];
int count = 0;

Scanner sc = new Scanner(System.in);
System.out.println("Enter a number or
-1 to stop:");

int input = sc.nextInt();
while (input != -1 && count<
numbers.length) {

        numbers [count] = input;
        count ++;
        input = sc.nextInt();
}
```

# Efficient Resizing Strategy

- **Double** the array size when full
- This reduces:
  - number of reallocations
  - total number of copied elements

**127 times**

```
int [] numbers = new int[1];
int count = 0;

for (int i = 0; i < 100; i++) {
        if (count == numbers.length) {
            // double the array size
            int [] newArray = new int[numbers.length*2];
            // Copy existing elements
            for (int j = 0; j < numbers.length; j++) {
                newArray [j] = numbers [j];
            }
            numbers = newArray;
        }
        // Add new element
        numbers [count] = i;
        count ++;
}
```

# Sorting an Array

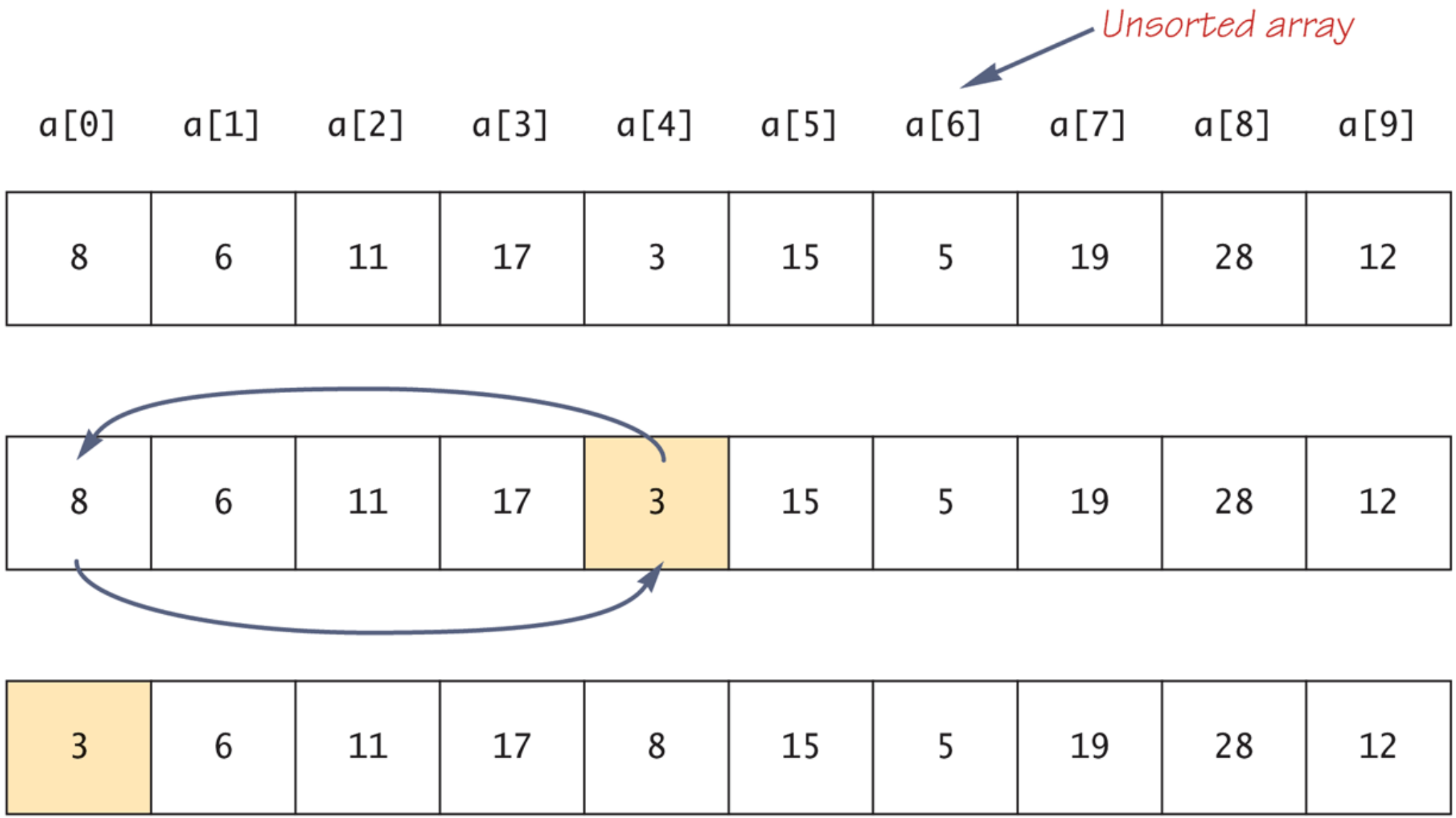

- Sorting is a fundamental task in computer science.
- Sorting **rearranges** elements within an array.
- A **selection sort** accomplishes this by using the following algorithm:

```
for (int index = 0; index < count; index++)
```

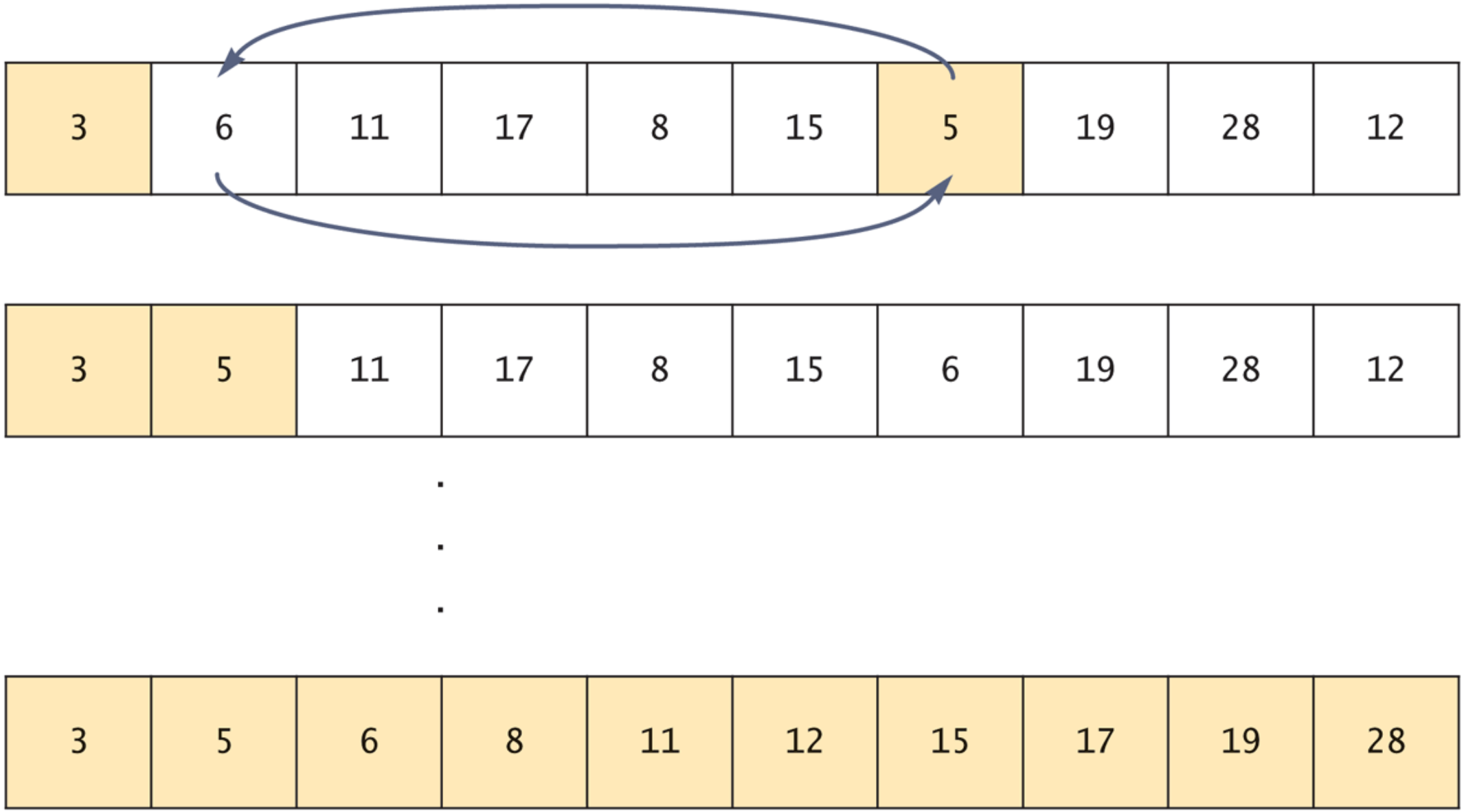
Find the smallest element from the unsorted portion and puts it at the start until the whole array is sorted.

- Selection sort is easy to understand and code, but there are much more efficient algorithms for sorting → Check ED lesson. This is advance topic for future units

## Display 6.10 Selection Sort



## Display 6.10 Selection Sort



# Multidimensional arrays


- What are multidimensional arrays?
  - Arrays with **more than one** index, the most popular is Two-dimensional arrays.
  - Useful for **structured** data storage such as tables, and matrices.
- Declaration syntax:

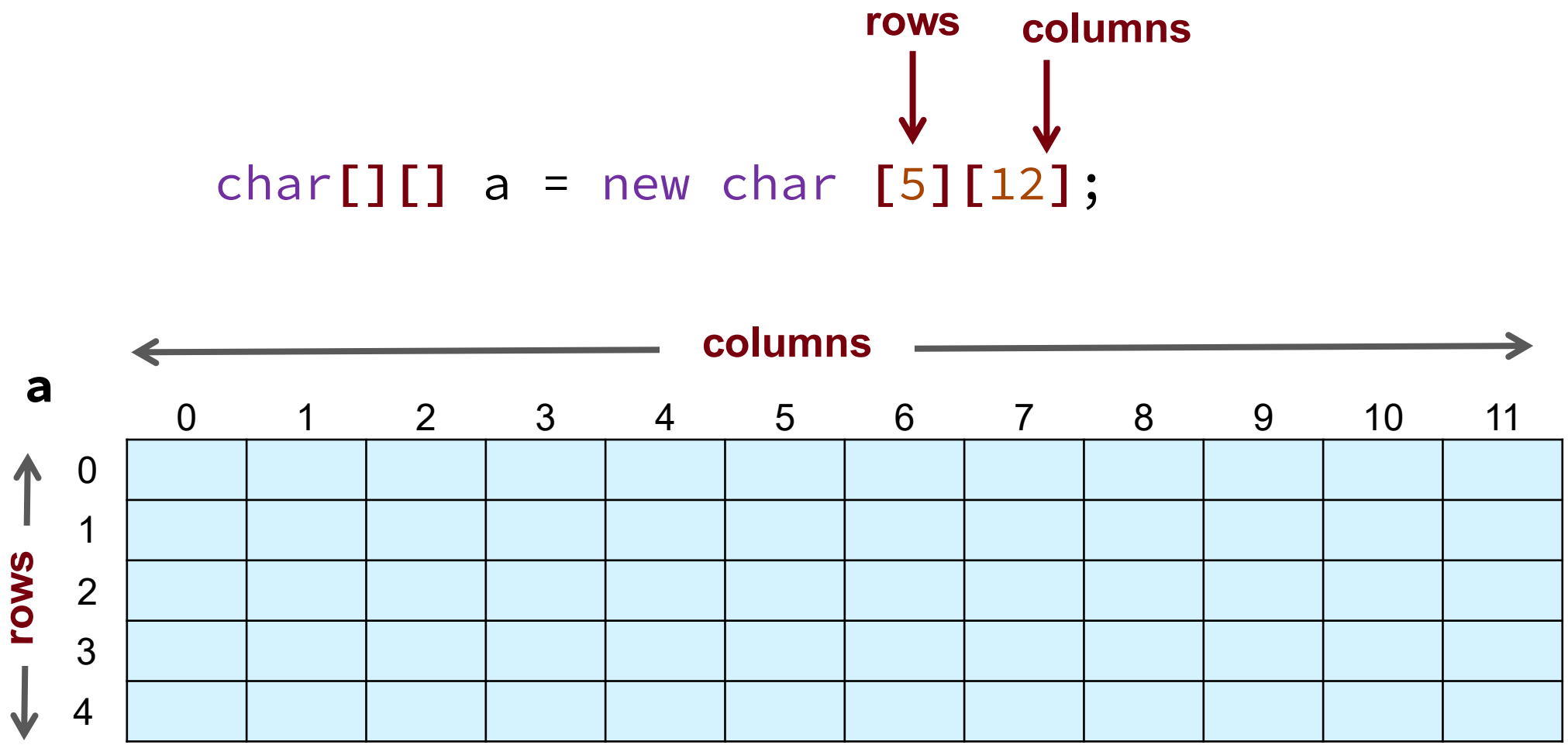
```
type[][] arrayName = new type [RowSize][ColumnSize];
```

- Example:

```
int[][] table = new int [100][10];
double[][][] Map = new double [20][20][20];
Person[][] = new Person[10][20];
```

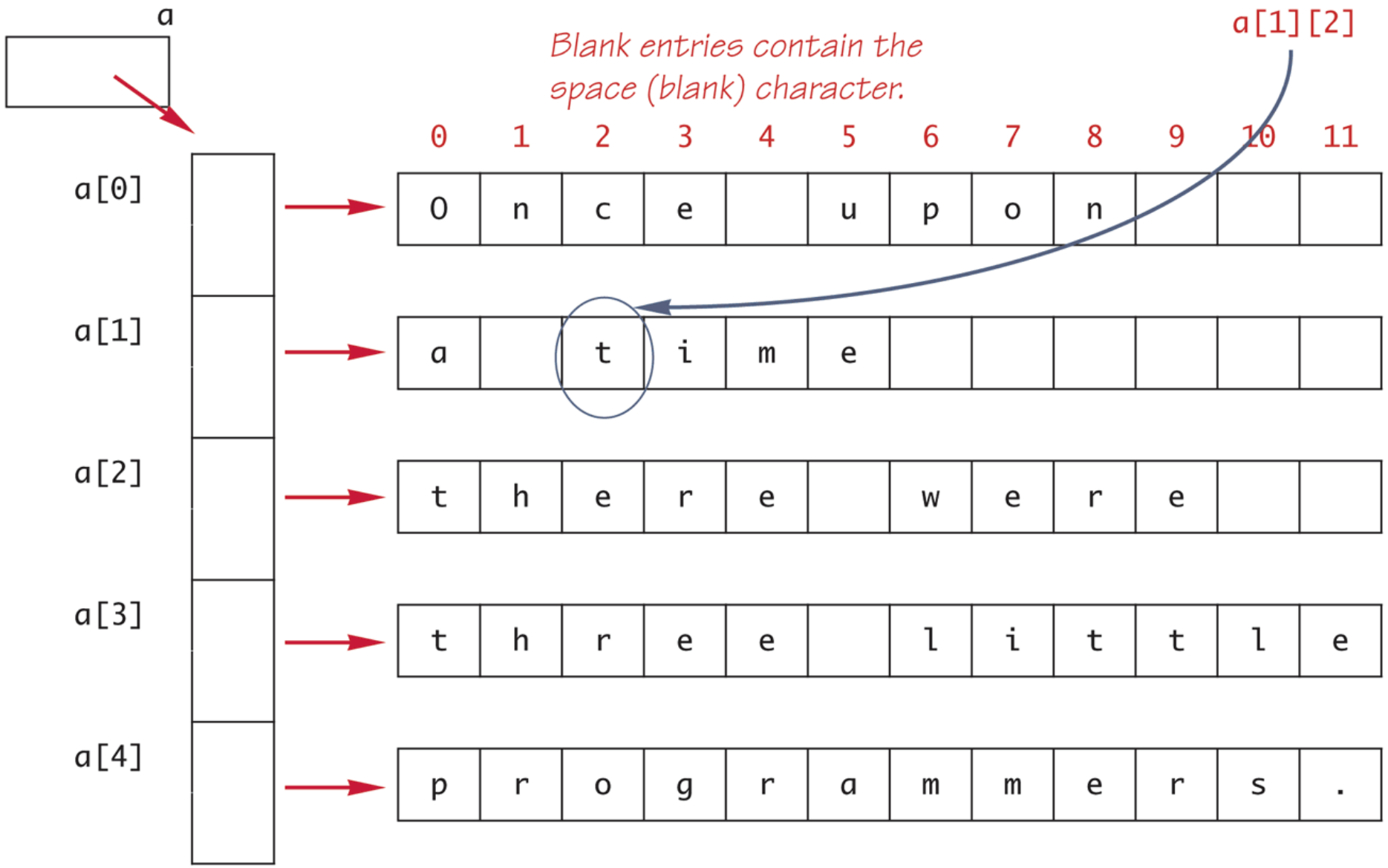
- Like a one-dimensional array, each element of a multidimensional array is just a variable of the base type.

# Two-Dimensional Arrays



```
char[][] a = new char[5][12];
```

*Code that fills the array is not shown.*



# Initializing Multidimensional Arrays

- **Initializing** example:

```
char[][] a = new char[][] {
    {'O', 'n', 'c', 'e', ' ', 'u', 'p', 'o', 'n', ' ', ' ', ' '},
    {'a', ' ', 't', 'i', 'm', 'e', ' ', ' ', ' ', ' ', ' ', ' '},
    {'t', 'h', 'e', 'r', 'e', ' ', 'w', 'e', 'r', 'e', ' ', ' '},
    {'t', 'h', 'r', 'e', 'e', ' ', 'l', 'i', 't', 't', 'l', 'e'},
    {'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'e', 'r', 's', '.'}};

for (int row = 0; row < 5; row++) {
    for (int column = 0; column < 12; column++)
        System.out.print(a[row][column]);
    System.out.println('|');
}
```

The Output will be:

```
Once upon    |
a time       |
there were   |
three little.|
programmers. |
```

# Arrays of arrays

- In Java, multidimensional arrays are really arrays of arrays.
- Array **dimensions**, from the previous char [][] a array example:
  - a.length → number of **rows** (5)
  - a[0].length → number of **columns** in first row (12)
- The “array of arrays” feature allows some **flexibility**.

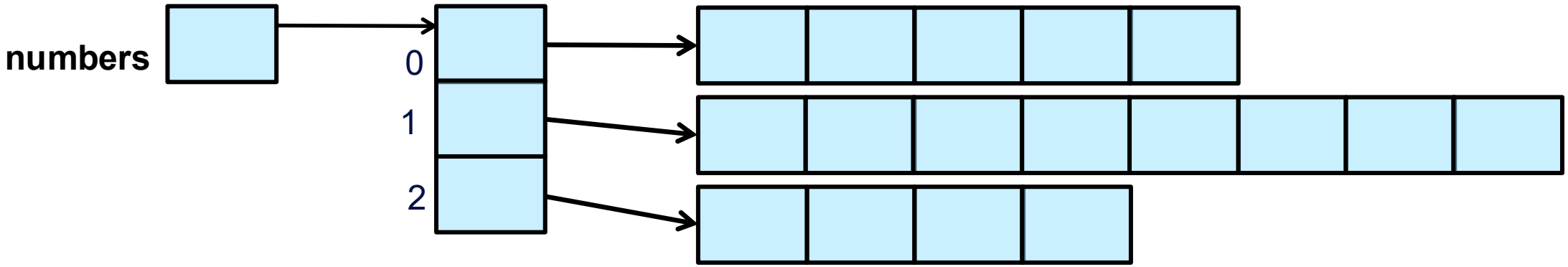
```
int[][] numbers = new int [3][5];
```

```
int[][] numbers = new int [3][];
numbers[0] = new int [5];
numbers[1] = new int [5];
numbers[2] = new int [5];
```

# Ragged Array

- Example:

```
int[][] numbers = new int [3][];
numbers[0] = new int [5];
numbers[1] = new int [8];
numbers[2] = new int [4];
```




- Ragged arrays Can:
  - Saves memory.
  - Increase complexity.

# Ragged Array

- Revisiting the previous example:

```
char[][] a = new char[][] {
    {'O', 'n', 'c', 'e', ' ', 'u', 'p', 'o', 'n'},
    {'a', ' ', 't', 'i', 'm', 'e'},
    {'t', 'h', 'e', 'r', 'e', ' ', 'w', 'e', 'r', 'e'},
    {'t', 'h', 'r', 'e', 'e', ' ', 'l', 'i', 't', 't', 'l', 'e'},
    {'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'e', 'r', 's', '.'}};

for (int row = 0; row < a.length; row++) {
    for (int column = 0; column < a[row].length; column++)
        System.out.print(a[row][column]);
    System.out.println('|');
}
```

```
for (char [] row : a)  {
    for(char letter : row ){
        System.out.print(letter);
    }
    System.out.println('|');
}
```

The Output will be:

```
Once upon|
a time|
there were|
three little|
programmers.|
```

# Multidimensional Arrays as Parameters


- Methods **returning** multidimensional arrays:

```
public double[][] generateMatrix () {...}
```

- Methods **accepting** multidimensional arrays:

```
public void printTable (int[][] table) {...}
```

- Passing 2D arrays as **arguments**:

```
int [][] numbers = {{85, 90, 78}, {76, 88, 91}};
printTable (numbers);
```

- Method that takes a **1D array** :

```
public void printRow (int[] row) {...}
printRow (numbers[0]); // Example of invoking the method
```


# Random Generator

# Random Generators


- Provides methods to generate pseudo-random numbers
- Commonly used in simulations, games, testing, etc.
- Uses a seed to produce a sequence of numbers in a predicted random way.
- Example
  - Random rand = new Random();
  - int num = rand.nextInt(100); // generates a random integer
  - boolean b = rand.nextBoolean(); // generates true or false randomly

# Important Methods


- `nextInt()` – returns a random int
- `nextInt(bound)` – returns int between 0 (inclusive) and bound (exclusive)
  - Example – nextInt(0,10) - generate a random number b/w 0 and 9 but not 10
- `nextDouble()` – returns a random double between 0.0 and 1.0
- `nextFloat()` – returns a random float between 0.0 and 1.0
- `nextBoolean()` – returns true or false randomly
- `setSeed(long seed)` – sets the seed for reproducibility
  - Why need reproducibility? To ensure that your code and my code generates same set of random numbers.
  - If you reset the generator and add the seed and run again, same set of random numbers will be generated.


# Live Coding
# (a variable no. of parameters & arrays)

# Live Coding Instruction – Order Engine


**Overview**

The system is an Order Engine that allows creating and editing food orders for multiple clients. It demonstrates:

- Arrays (1D and 2D)
- Partially filled arrays with counters
- Resizable arrays (doubling strategy)
- Variable number of parameters (varargs)
- Encapsulation across multiple classes

The system consists of three main components:

- Menu (enum)
- Client (class)
- OrderEngine (class with main method)

Each order corresponds to exactly one client, and the order index refers to the client's index in the client array.

# Live Coding Instruction – Order Engine

**Menu Enum** (Menu.java)

*Purpose*: Represents the available meals and drinks that can be ordered by clients.

*Responsibilities*:

- Define a fixed set of menu items (e.g. CHEESE).
- Be used as elements in order arrays.

# Live Coding Instruction – Order Engine

**Client Class** (Client.java)

*Purpose*: Represents a single client and their associated order.

*Class Signature*: public class Client

*Members*:

- Variables: String clientName, int orderIndex, and Menu[] orders.
- Methods: constructors (initialize variables), getters, setters, print client information, and add/remove meals to the orders (using a variable number of parameters).

*Behaviours of Core Methods:*

- Add meals:
  - Accepts a variable number of Menu items.
  - Adds each item to the orders array.
  - If the array is full, resizes it using a doubling strategy.
- Remove meals:
  - Accepts a variable number of Menu items.
  - Removes matching items from the client's order.
  - Shifts remaining elements to keep the array compact.

# Live Coding Instruction – Order Engine

**OrderEngine Class** (OrderEngine.java)

*Purpose*: Manages multiple clients and coordinates order creation and modification.

*Class Signature*: public class OrderEngine

*Members*:

- Variables: an array of clients and an orderIndex used as a counter for new orders, which *avoids* accessing preallocated but uninitialized clients.
- Methods: constructors (preallocate for one client and increase by *2 when reaching the maximum capacity), getters, print all client information, print a 2D array for all orders (each row for an order/client and each column for a meal), create an order (increase orderIndex by 1), edit (add/remove) the order of a client (identified by the order index), and a main() method.

*Behaviours of Core Methods*:

- Create a New Order:
  - Creates a new Client.
  - Adds it to the client array.
  - Resizes the array by doubling if capacity is reached.
  - Increments orderIndex.

# Live Coding Instruction – Order Engine

*Behaviours of Core Methods*:

- Add Meals to an Order:
  - Identifies the client using the provided index.
  - Adds menu items using a variable no. of parameters.
- Remove Meals from an Order (first occurrence).
- Main() Method:
  - Creating orders.
  - Adding and removing menu items.
  - Printing clients.
  - Printing the 2D order table.

See detailed instructions in Ed Workspace/week6.

# Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 6)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 3)
- Array (accessible on 14-02-2024) Oracle's Java Documentation. Available at: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html.
- Arrays (accessible on 14-02-2024) Java 7 Documentation. Available at: https://docs.oracle.com/javase/specs/jls/se7/html/jls-10.html.

THE UNIVERSITY OF MELBOURNE

# See you next time!




The University of Melbourne (Australian University) PRV12150/CRICOS 00116K